

no java before code

◆ EXTRACTING THE PREDICTION VALUE

```
result = response.getBody()
                .get("prediction")
                .toString();
```

This line executes only if the previous condition was true.

Remember:

```
if (response.getBody() != null &&
    response.getBody().get("prediction") != null)
```

So Java already confirmed:

- ✓ Response body exists
- ✓ Prediction exists

Now we can safely read the prediction value.

Flow

Response Exists

↓

Prediction Exists

↓

Read Prediction

↓

Store In result

◆ WHAT IS HAPPENING HERE?

Suppose Flask returned:

```
{
  "prediction": "SPAM"
}
```

After Spring converts JSON into Map:

```
{
  prediction=SPAM
}
```

Now Java executes:

```
response.getBody()
```

Result:

```
{
  prediction=SPAM
}
```

This returns the Map.

Visual

response

↓

Body

↓

Map

↓

prediction = SPAM

◆ GETTING THE VALUE

```
.get("prediction")
```

This means:

"Find the key named prediction and return its value."

Example Map:

```
{
  prediction=SPAM
}
```

Java searches:

Key?

```
prediction
```

Found?

✓ Yes

Returns:

```
SPAM
```

Think Of A Dictionary

Dictionary

```
prediction → SPAM
```

When you ask:

```
get("prediction")
```

Java gives:

```
SPAM
```

◆ WHAT DOES `get()` RETURN?

Many beginners think:

```
get("prediction")
```

returns a `String`.

Not always.

Actually:

```
Map
```

stores values as `Objects`.

Therefore:

```
get()
```

returns:

```
Object
```

Internally:

```
Object value =  
response.getBody().get("prediction");
```

In our case:

```
value = "SPAM";
```

But Java still treats it as:

```
Object
```

◆ WHY USE `toString()` ?

```
.toString()
```

converts the returned object into text.

Remember:

```
result
```

is a `String` variable.

```
String result;
```

Therefore we need a String.

Suppose:

```
Object value = "SPAM";
```

Then:

```
value.toString();
```

returns:

```
"SPAM"
```

which is a String.

Visual

Object

↓

toString()

↓

String

◆ WHAT IS toString() ?

Every Java object inherits a method called:

```
toString()
```

from the Object class.

Purpose:

Convert an object into text.

Example

```
Integer number = 100;  
number.toString();
```

returns:

```
"100"
```

Another Example

```
Boolean flag = true;  
flag.toString();
```

returns:

```
"true"
```

Similarly:

```
"SPAM".toString();
```

returns:

```
"SPAM"
```

◆ FINAL RESULT OF THE LINE

Starting value:

```
result = "ERROR";
```

Response:

```
{
  prediction=SPAM
}
```

Execution:

```
result =
response.getBody()
    .get("prediction")
    .toString();
```

Step 1

```
response.getBody()
```

↓

```
{
  prediction=SPAM
}
```

Step 2

```
get("prediction")
```

↓

```
SPAM
```

Step 3

```
toString()
```

↓

```
"SPAM"
```

Step 4

Assign to result

```
result = "SPAM";
```

Final value:

```
result
```

↓

```
SPAM
```

◆ WHY STORE IT IN result?

Because later we need it inside the DTO.

Without storing:

```
return new EmailResponseDTO(
    response.getBody()
        .get("prediction")
        .toString(),
    1.0
);
```

would work.

But it becomes harder to read.

Using:

```
String result
```

makes the code cleaner.

Flow

Prediction

↓

Store In result

↓

Reuse Later

◆ RETURNING THE RESPONSE DTO

```
return new EmailResponseDTO(result, 1.0);
```

This is the final successful return statement.

The service has completed its work.

Flow

Receive Email

↓
 Call Flask
 ↓
 Get Prediction
 ↓
 Build DTO
 ↓
 Return DTO

◆ WHAT DOES return MEAN?

The keyword:

```
return
```

means:

"Send a value back to whoever called this method."

Remember:

```
public EmailResponseDTO analyze(String content)
```

The return type is:

```
EmailResponseDTO
```



Therefore Java expects:

```
return someEmailResponseDTO;
```

◆ WHO CALLED THIS METHOD?

Earlier:

```
emailService.analyze(...)
```



IMP.

was called inside EmailController.

Visual

EmailController

↓

analyze()

↓

EmailService

After analysis completes:

EmailService

↓

returns DTO

↓

EmailController

◆ CREATING A NEW DTO

```
new EmailResponseDTO(result, 1.0);
```

This creates a brand-new DTO object.

Think of a DTO as a container.

EmailResponseDTO

|— result

|— confidence

Example

```
new EmailResponseDTO(
  "SPAM",
  1.0
);
```

Creates:

```
{
  result : "SPAM",
  confidence : 1.0
}
```

◆ WHAT IS result HERE?

Suppose Flask predicted:

```
SPAM
```

Then:

```
result = "SPAM";
```

So:

```
new EmailResponseDTO(
  result,
  1.0
)
```

becomes:

```
new EmailResponseDTO(
  "SPAM",
  1.0
)
```


◆ WHAT IS 1.0 ?

1.0

is a double value.

Currently your Flask API only returns:

```
{
  "prediction": "SPAM"
}
```

It does NOT return confidence.

Therefore:

1.0

is being used as a placeholder.

Meaning:

"Prediction succeeded."

Not necessarily:

"100% confidence."

This is important.

Many students misunderstand this.

Current Project

Prediction:

SPAM

Confidence:

1.0

Hardcoded Value

Future Version

Flask may return:

```
{
  "prediction": "SPAM",
  "confidence": 0.94
}
```

Then you could store:

0.94

instead of:

1.0

◆ WHY IS 1.0 USED EVEN THOUGH WE ARE ALREADY IN THE SUCCESS BLOCK?

This is a very good observation.

Many beginners think:

"If we already reached this line, doesn't that automatically mean success?"

Yes.

The code has already verified:

```
response.getBody() != null
```

and

```
response.getBody().get("prediction") != null
```

Therefore we know Flask successfully returned a prediction.

So why write:

```
new EmailResponseDTO(result, 1.0);
```

The answer is:

Because the DTO requires two values.

Look at the constructor:

```
EmailResponseDTO(
    result,
    confidence
)
```

The first value stores the prediction.

The second value stores confidence.

Even though we know the prediction succeeded, the DTO still expects a confidence value.

Since Flask currently does not send confidence, the developer chose:

1.0

as a placeholder value.

Think of it like this:

Prediction Available?

↓

Yes

↓

Need Confidence?

↓

Not Available

↓

Use Default Value

↓

1.0

So:

1.0

does not prove success.

The success was already proven by the previous checks.

1.0

is simply filling the confidence field because the DTO requires it. 

◆ WHAT DOES THE CONTROLLER RECEIVE?

Service returns:

```
new EmailResponseDTO(
    "SPAM",
    1.0
);
```

Controller receives:

EmailResponseDTO response

containing:

```
{
  result:"SPAM",
  confidence:1.0
}
```

Then Spring converts it into JSON.

Final API Response:

```
{
  "result":"SPAM",
  "confidence":1.0
}
```

Swagger receives this response.

◆ COMPLETE SUCCESS FLOW

User enters email

↓

Swagger sends request



Controller receives request



Controller calls Service



Service creates JSON



RestTemplate sends request



Flask receives email



ML Model predicts SPAM



Flask returns prediction



Spring receives response



Prediction extracted



DTO created



Controller receives DTO



Spring converts DTO to JSON



Swagger displays result

◆ SUMMARY

```
result =  
response.getBody()  
    .get("prediction")  
    .toString();
```

extracts the prediction value from the Flask response and converts it into a String.

```
return new EmailResponseDTO(result, 1.0);
```

creates the final DTO object and returns it to the controller.

The prediction success was already confirmed by the previous safety checks.

1.0

is simply a placeholder confidence value because the DTO requires a confidence field but Flask currently returns only the prediction.

The next section should continue with:

```
catch (Exception e) {
    return new EmailResponseDTO("ERROR", 0.0);
}
```



because this explains exception handling, application failures, fallback responses, error recovery, and why the API does not crash even when Flask is unavailable.